



Software Factory en la Era de la IA

Metricas

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Dora Metrics

Visión General

Las métricas DORA (DevOps Research and Assessment), popularizadas por el libro *Accelerate* de Nicole Forsgren, Jez Humble y Gene Kim, son las 4 métricas clave que predicen el desempeño de los equipos de software y la capacidad de entregar valor de negocio.

Las 4 Key Metrics

Métrica	Qué mide	Fórmula
Deployment Frequency	Frecuencia de despliegues a producción	Despliegues / período
Lead Time for Changes	Tiempo desde commit hasta producción	Timestamp(producción) – Timestamp(commit)
Change Failure Rate	% de cambios que causan fallo en producción	Cambios con fallo / Total cambios × 100
Mean Time to Recovery (MTTR)	Tiempo promedio para recuperarse de fallo	Sum(recovery times) / N° incidents

Niveles de Desempeño

Nivel	Deploy Freq	Lead Time	Change Failure Rate	MTTR
Elite	On-demand (múltiples/día)	< 1 hora	0-15%	< 1 hora
High	Entre semanal y mensual	1 día - 1 semana	16-30%	< 1 día
Medium	Entre mensual y semanal	1 semana - 1 mes	16-30%	< 1 día
Low	Entre semanal y mensual	1-6 meses	16-30%	1 día - 1 semana

Cómo Medir Cada Métrica

Deployment Frequency

Deployment Frequency = N° deploys exitosos / Período de tiempo

Ejemplo: 45 deploys / mes = ~2.25 deploys/día

Fuentes de datos: GitHub Actions, Jenkins, GitLab CI, ArgoCD

Buenas prácticas:

- Contar solo deploys a producción (no staging)

- Incluir deploys automáticos y manuales
- Usar mediana en vez de media para evitar outliers

Lead Time for Changes

Lead Time = $P50(\text{Production Timestamp} - \text{First Commit Timestamp})$

Ejemplo: $P50 = 3.2$ horas → mediana del tiempo de entrega

Fuentes de datos: Git logs + deployment timestamps, CI/CD pipeline metrics

Consideraciones:

- Medir desde el primer commit del changeset, no desde el push
- Incluir tiempo de code review y CI
- Segmentar por tipo de cambio (feature, bugfix, hotfix)

Change Failure Rate

CFR = $(\text{Deploys con rollback/fallo detectado} / \text{Total deploys}) \times 100$

Ejemplo: 5 fallos en 45 deploys = 11.1% CFR

Definición de "fallo":

- Rollback inmediato
- Hotfix dentro de 24 horas
- Degradación de servicio detectada por monitoreo

Mean Time to Recovery

MTTR = $\Sigma(\text{Tiempo de recuperación por incidente}) / N^{\circ} \text{ incidentes}$

Ejemplo: $(45\text{min} + 120\text{min} + 30\text{min}) / 3 = 65 \text{ min}$

Fuentes de datos: PagerDuty, Opsgenie, incident management tools

Dashboard de Ejemplo

DORA DASHBOARD – Q3 2026			
Deploy Frequency	2.3/día	[ELITE]	
Lead Time (P50)	4.2h	[ELITE]	
Change Fail Rate	12%	[ELITE]	
MTTR (P50)	38min	[ELITE]	

| Tendencia: ↗ Deploy Freq ↗ Lead Time (↓ mejor) |
| ↘ CFR (↓ mejor) ↘ MTTR (↓ mejor) |

Herramientas de Medición

Herramienta	Tipo	Métricas DORA
DORA metrics (Google)	Herramienta nativa	Las 4 key metrics
Accelerate plugin	Jenkins plugin	Deploy freq, lead time
GitLab Analytics	Built-in	Todas las 4
Faros AI	Plataforma	DORA + extras
LinearB	Plataforma	DORA + flow metrics
改 (改)	OSS	Custom DORA dashboard

Factores que Impulsan el Alto Desempeño

Según el State of DevOps Report, los equipos elite se diferencian por:

1. **Trunk-based development** — No feature branches largos
2. **CI/CD completo** — Deploy on demand
3. **Feature flags** — Desacoplar deploy de release
4. **Automated testing** — Suite de tests confiables
5. **Loose coupling** — Arquitectura que permite despliegues independientes
6. **Generative culture** — Aprendizaje post-incidente, no blame

Métricas Complementarias

Las DORA metrics son necesarias pero no suficientes. Complementar con:

- Velocity y Throughput para capacidad del equipo
- Tasa de Defectos para calidad percibida
- Salud del Equipo para sostenibilidad
- Satisfacción del Cliente para valor real

Anti-Patrones

Anti-Patrón	Consecuencia
Optimizar solo deployment frequency	Feature factory, código de baja calidad
Ignorar change failure rate	Velocidad sin calidad = más incidentes
No medir MTTR	No saber si el equipo puede recuperarse
Comparar equipos entre sí	Competencia tóxica en vez de mejora

Referencias

- DevOps Filosofía
- CI/CD Pipeline
- Deployment Frequency en detalle
- Change Failure Rate
- Mean Time to Recovery
- Balanced Scorecard

Velocity Throughput

Visión General

Velocity y throughput son métricas de capacidad que intentan cuantificar cuánto trabajo entrega un equipo. Aunque son ampliamente usadas, tienen limitaciones importantes que deben entenderse para usarlas correctamente.

Velocity (Story Points)

Definición

Velocity es la cantidad de story points completados por un equipo en un sprint (típicamente 2 semanas).

$$\text{Velocity} = \Sigma \text{ Story Points del sprint} / \text{Nº sprints}$$

Ejemplo: $(21 + 18 + 24 + 20 + 22) / 5 = 21 \text{ points/sprint}$ (promedio)

Estimación por Complejidad

Points	Complejidad	Tiempo estimado
1	Trivial	< 1 hora
2	Simple	1-3 horas
3	Moderada	3-8 horas
5	Compleja	1-2 días
8	Muy compleja	2-3 días
13	Épica	> 3 días (partir)

Limitaciones de Velocity

Limitación	Problema
No es comparable entre equipos	20 points de equipo A $\neq$ 20 de equipo B
No mide valor	Entregar 20 points de features innecesarias = desperdicio
Gaming	Los equipos aprenden a inflar estimaciones
Estimación imprecisa	Planning poker es inherentemente inexacto
No refleja calidad	Velocity no distingue código limpio de código frágil

Throughput (Opción Alternativa)

Definición

Throughput es el número de items completados por período, sin importar su tamaño.

$\text{Throughput} = \text{Nº items completados} / \text{Período}$

Ejemplo: 14 stories completados / 2 semanas = 7 stories/sprint

Throughput vs Velocity

Aspecto	Velocity	Throughput
Base	Story points (estimación)	Conteo de items
Estimación requerida	Sí	No
Comparabilidad	Solo intra-equipo	Casi comparable
Precisión	Depende de la estimación	Simple de medir
Gaming risk	Alto	Bajo

Little's Law

$\text{Throughput} = \text{WIP} / \text{Cycle Time}$

Si WIP promedio = 5 items y Cycle Time = 2 días
→ $\text{Throughput} = 5 / 2 = 2.5 \text{ items/día}$

Esto conecta directamente con Cycle Time y la gestión de WIP en Kanban.

Métricas Superiores a Velocity

Métrica	Ventaja	Referencia
Cycle Time	Mide velocidad real del flujo	Cycle time
Flow Efficiency	Identifica tiempo muerto	Cycle time
DORA Metrics	Outcome-based, no output-based	Dora metrics
OKR Progress	Alineado con valor de negocio	Business value metrics

Mejores Prácticas

Usar Velocity Correctamente

- Solo para forecasting** — Predecir cuánto cabrá en sprints futuros
- Nunca para comparar equipos** — Cada equipo tiene su escala

3. **Tendencia, no absoluto** — Importa si sube o baja, no el número
4. **Complementar con outcome metrics** — Velocity + satisfacción + quality

Capacity Planning con Velocity

Capacidad sprint N+1 = Velocity promedio × Factor de disponibilidad

Ejemplo: 21 pts × 0.85 (vacaciones, feriados) = ~18 points estimados

Factores de disponibilidad:

- Vacaciones programadas
- Feriados
- Actividades de soporte/mantenimiento
- Onboarding de nuevos miembros
- Actividades técnicas (tech debt, refactoring)

Dashboard de Ejemplo

VELOCITY & THROUGHPUT – ÚLTIMOS 6			
Sprint	Velocity	Throughput	Cycle Time
S-21	22	11	3.2d
S-22	18	9	4.1d
S-23	24	12	2.8d
S-24	20	10	3.5d
S-25	21	11	3.0d
S-26	23	12	2.9d
Avg:	21.3	10.8	3.25d
Trend:	→	→	↘ (mejor)

Conexiones

- Cycle Time — Métrica superior para velocidad de flujo
- DORA Metrics — Framework completo de medición
- Valor de Negocio — Conectar capacidad con valor
- Salud del Equipo — Velocity puede indicar burnout
- Scrum — Contexto donde se usa velocity
- Kanban — Throughput como alternativa

Cycle Time

Visión General

Cycle Time y Lead Time son métricas de flujo que miden la velocidad real de entrega. A diferencia de velocity, miden tiempo transcurrido, no esfuerzo estimado, lo que las hace más objetivas y accionables.

Definiciones

Lead Time

Tiempo total desde que se solicita algo hasta que se entrega al usuario.

Lead Time = Timestamp(Entrega al usuario) - Timestamp(Solicitud)

Ejemplo: Solicitud martes 9am → Entrega viernes 3pm = 3.25 días

Componentes del Lead Time:

- Tiempo de espera (backlog prioritization)
- Tiempo de desarrollo activo
- Tiempo de code review
- Tiempo de CI/CD
- Tiempo de deployment
- Tiempo de validación post-deploy

Cycle Time

Tiempo desde que comienza el trabajo activo hasta que se entrega.

Cycle Time = Timestamp(Entrega) - Timestamp(Primer commit)

Ejemplo: Primer commit lunes 10am → Deploy miércoles 2pm = 2.1 días

Diferencia Clave

|←—— Lead Time ——→|
| Wait |← Cycle Time →|
| Backlog| Dev | Review | CI | Deploy |

Fórmulas y Métricas Relacionadas

Percentiles

P50 (Mediana) = 50% de los items se completan en este tiempo
P85 = 85% de los items se completan en este tiempo
P95 = 95% de los items se completan en este tiempo

Ejemplo: P50 = 2.5d, P85 = 5d, P95 = 12d
→ El 50% de las features se entregan en menos de 2.5 días

Usar percentiles en vez de promedio es fundamental — el promedio oculta la variabilidad.

Flow Efficiency

Flow Efficiency = (Tiempo activo / Cycle Time total) × 100

Ejemplo: 8h activo / 48h total = 16.7%

Un flow efficiency del 16.7% significa que el 83% del tiempo el trabajo está esperando. Los equipos elite típicamente alcanzan 30-50%.

Throughput (conectado)

Throughput = WIP / Cycle Time (Little's Law)

Si WIP promedio = 8 y Cycle Time = 3 días
→ Throughput = 8/3 = 2.67 items/día

Ver Velocity throughput para más detalles.

Cómo Medir

Herramientas

Herramienta	Cómo mide	Costo
Jira	Built-in reports (Control Chart)	Licencia
GitHub Projects	Custom fields + queries	Gratis
Linear	Built-in metrics	SaaS
Jellyfish	Engineering analytics	Enterprise
Pluralsight Flow	Git-based analysis	SaaS

Medición Manual (Git)

```
# Cycle time entre PR merge y deploy
git log --format="%H %ai" --merges | \
```

```
while read hash date; do
  # Calcular diferencia con timestamp de deploy
done
```

Data Points Necesarios

Evento	Fuente	Qué captura
Story creada	Jira/GitHub	Inicio del Lead Time
Commit inicial	Git	Inicio del Cycle Time
PR abierto	GitHub	Inicio de review
PR mergeado	GitHub	Fin de review
Deploy exitoso	CI/CD	Fin del Cycle Time

Estrategias de Mejora

Reducir Cycle Time

Estrategia	Impacto	Esfuerzo
Reducir WIP limit	Alto	Bajo
Trunk-based development	Alto	Medio
CI/CD automatizado	Alto	Medio
Code review same-day	Medio	Bajo
Feature flags	Alto	Medio
Smaller PRs	Medio	Bajo

Reducir Lead Time

Estrategia	Impacto	Esfuerzo
Priorización continua	Alto	Bajo
Eliminar handoffs	Alto	Alto
Cross-functional teams	Alto	Alto
Automatizar testing	Medio	Medio

Dashboard de Ejemplo

CYCLE TIME DASHBOARD – ÚLTIMOS 30 DÍAS

P50: 2.3 días 
P85: 4.8 días 
P95: 8.1 días 

Flow Efficiency: 28% (meta: 40%)

Por tipo:

Bugfix: P50 = 0.8d 
Feature: P50 = 3.2d 
Tech Debt: P50 = 1.5d 
Hotfix: P50 = 0.3d 

Conexiones

- DORA Metrics — Lead Time es una de las 4 key metrics
- Velocity y Throughput — Cycle Time es superior a velocity
- Value Stream Mapping — Identificar desperdicios en el flujo
- Kanban — WIP limits reducen cycle time
- CI/CD Pipeline — Automatizar reduces tiempos
- Tasa de Defectos — Código rápido sin calidad es insostenible
- Observabilidad — Medir tiempos end-to-end

Defect Rate

Visión General

La tasa de defectos mide la calidad del software desde la perspectiva de errores y fallos. Es un indicador lagging pero esencial para entender la salud técnica del producto y la efectividad de las prácticas de testing.

Métricas Principales

Defect Density (Densidad de Defectos)

Defect Density = $\frac{\text{Nº defectos}}{\text{KLOC (Kilo Lines of Code)}}$
o $\frac{\text{Nº defectos}}{\text{Function Point}}$

Ejemplo: 15 defectos / 50 KLOC = 0.3 defectos/KLOC

Benchmarks:

Calidad	Defectos/KLOC
Excelente	< 0.1
Buena	0.1 - 0.5
Aceptable	0.5 - 1.0
Problemática	> 1.0

Escaped Defects (Defectos que Escapan)

Escaped Defects Rate = $\frac{\text{Defectos encontrados en producción}}{\text{Total defectos}} \times 100$

Ejemplo: 8 defectos en prod / 45 defectos totales = 17.8%

Segmentación por fase de detección:

Fase	Costo relativo de fix
Requirements	1x
Design	5x
Code (review)	10x
Testing	20x
Production	100x

Bug Backlog

Bug Backlog = Total bugs abiertos – Total bugs cerrados

Net Bug Growth = Nuevos bugs / Cerrados por período

Indicadores saludables:

- Ratio cerrados/abiertos > 1 → backlog se reduce
- Ratio cerrados/abiertos < 1 → backlog crece (alerta)
- Bugs > 90 días = tech debt crítico

Mean Time to Detection (MTTD)

MTTD = $\Sigma(\text{Tiempo desde introducción hasta detección}) / N^{\circ} \text{ defectos}$

Ejemplo: Promedio 4.2 días para detectar defectos en producción

Mean Time to Resolution (MTTR de bugs)

MTTR_bug = $\Sigma(\text{Tiempo desde reporte hasta fix deployado}) / N^{\circ} \text{ bugs}$

Ejemplo: P50 = 3 días, P85 = 12 días

Clasificación de Defectos

Severidad	Descripción	SLA Típico
S1 - Crítico	Sistema caído, data loss	4 horas
S2 - Mayor	Funcionalidad core rota	1 día
S3 - Moderado	Workaround disponible	1 semana
S4 - Menor	Cosmético, baja prioridad	Next sprint

Dashboard de Ejemplo

DEFECT METRICS DASHBOARD – Q3 2026				
Defect Density:	0.22/KLOC		OK	
Escaped Defects:	12%		OK	
Bug Backlog:	23 abiertos		↗ creciendo	⬆
MTTD (P50):	1.8 días		OK	
MTTR Bug (P50):	2.1 días		OK	

Por severidad:	S1:0 S2:2 S3:8 S4:13
Bugs > 30 días:	5 (22% del backlog)
Ratio cierre:	1.3 cerrados/abiertos

Estrategias de Mejora

Prevenir Defectos

Estrategia	Impacto	Referencia	
TDD	Alto	[[../04-Practicas/03-test-driven-development\	TDD]]
BDD	Alto	[[../04-Practicas/04-behavior-driven-development\	BDD]]
Pair programming	Medio	[[../04-Practicas/05-pair-programming\	Pair Programming]]
Code review	Alto	[[../04-Practicas/06-code-review-best-practices\	Code Review]]
Shift-left testing	Alto	[[../01-Fundamentos/08-shift-left\	Shift Left]]

Detectar Temprano

Estrategia	Herramienta
Static analysis	SonarQube, ESLint
Mutation testing	Pitest, Stryker
Property-based testing	Hypothesis, fast-check
Contract testing	Pact

Gestionar el Backlog

1. **Weekly bug triage** — Revisar bugs abiertos cada semana
2. **Bug debt budget** — Asignar % del capacity a bugs
3. **Auto-close stale bugs** — Bugs > 180 días sin activity
4. **Root cause analysis** — Para S1/S2, no solo hotfix

Métricas Complementarias

Métrica	Qué complementa	
[[05-code-coverage\	Code Coverage]]	Prevención de defectos
[[06-technical-debt-metrics\	Tech Debt]]	Causa raíz de muchos defectos
[[12-change-failure-rate\	Change Failure Rate]]	Defectos introducidos por cambios

Métrica	Qué complementa	
[[08-customer-satisfaction\	CSAT]]	Impacto percibido por el usuario

Referencias

- TDD
- BDD
- Code Review
- Herramientas de Testing
- Shift Left
- Deuda Técnica
- DORA Metrics

Code Coverage

Visión General

Code coverage mide qué porcentaje del código fuente es ejecutado por los tests. Es una métrica útil pero frecuentemente malinterpretada — coverage alto no garantiza calidad, y coverage bajo es una señal de alerta.

Tipos de Coverage

Statement Coverage (Cobertura de Sentencias)

$\text{Statement Coverage} = (\text{Sentencias ejecutadas} / \text{Total sentencias}) \times 100$

Ejemplo: 850 sentencias ejecutadas / 1000 totales = 85%

La métrica más básica. Indica qué % del código fue "pisado" por tests.

Branch Coverage (Cobertura de Ramas)

$\text{Branch Coverage} = (\text{Ramas ejecutadas} / \text{Total ramas}) \times 100$

Ejemplo: if/else tiene 2 ramas. Si solo se testea true → 50%

Más exhaustiva que statement coverage — asegura que tanto true como false se testeen.

Function Coverage (Cobertura de Funciones)

$\text{Function Coverage} = (\text{Funciones llamadas} / \text{Total funciones}) \times 100$

Verifica que cada función sea invocada al menos una vez.

Mutation Coverage

$\text{Mutation Score} = (\text{Mutantes killed} / \text{Total mutantes}) \times 100$

Ejemplo: 200 mutantes creados, 160 killed → 80% mutation score

Mutation testing introduce cambios artificiales (mutantes) en el código y verifica que los tests fallen. Es la métrica más rigurosa de calidad de tests.

Resumen Comparativo

Tipo	Qué mide	Dificultad	Valor
Statement	Líneas ejecutadas	Bajo	Básico
Branch	Decisiones ejecutadas	Medio	Bueno
Function	Funciones invocadas	Bajo	Básico
Mutation	Efectividad de tests	Alto	Excelente

Testing Pyramid Metrics

```

      /  E2E Tests  \      → 10% del total, lentos
     /  Integration  \      → 20-30%, moderate
    /   Unit Tests   \      → 60-70%, rápidos
  
```

Capa	Coverage meta	Velocidad	Costo
Unit	> 80%	< 1ms/test	Bajo
Integration	> 60%	< 100ms/test	Medio
E2E	> 40% critical paths	> 1s/test	Alto

Coverage por Capa

Overall Coverage = (Unit lines + Integration lines + E2E lines) / Total lines

Pero cuidado: la superposición importa.
Un path testeado en E2E Y en unit → contar una vez.

Cobertura por Herramienta

Herramienta	Lenguaje	Tipo	Mutation
Istanbul/nyc	JavaScript	Statement/Branch	No
JaCoCo	Java	Statement/Branch	No
Coverage.py	Python	Statement/Branch	No
Pitest	Java	—	Sí
Stryker	JS/TS/C#	—	Sí
mutmut	Python	—	Sí

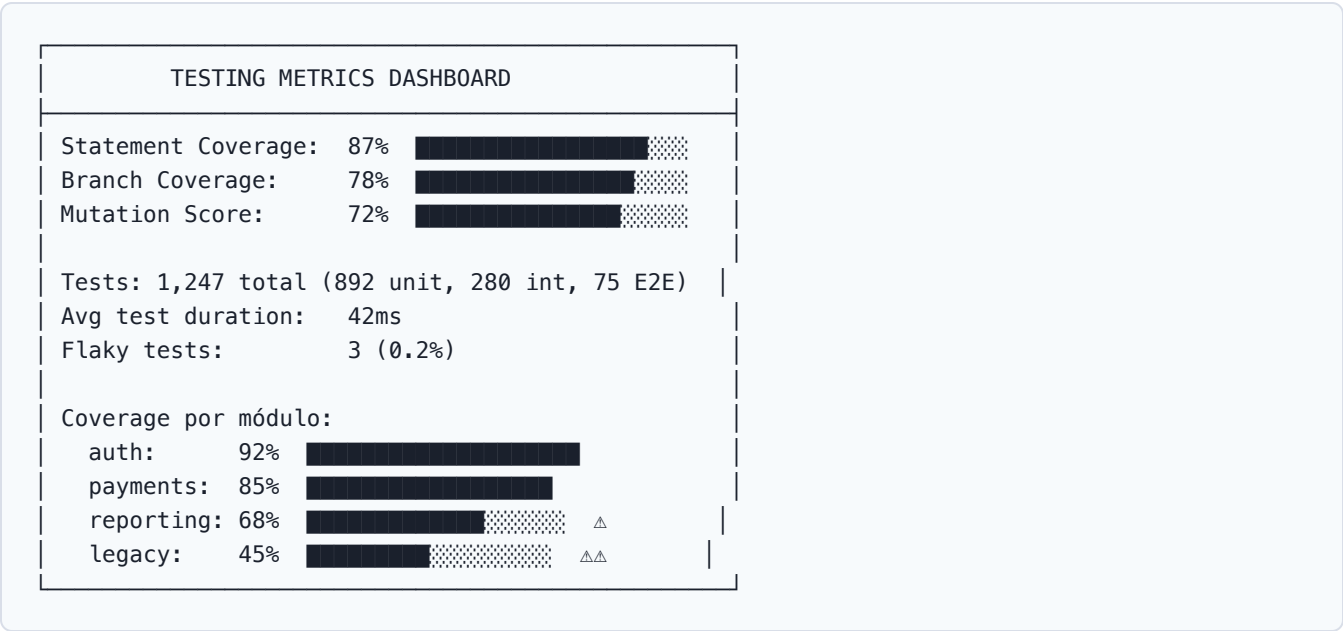
Targets de Coverage

Nivel	Statement	Branch	Mutation	Cuándo usar
Mínimo	60%	50%	—	Proyectos legacy
Estándar	80%	70%	—	Proyectos activos
Alto	90%	85%	70%	Critical systems
Máximo	95%+	90%+	80%+	Safety-critical

Anti-Patrones

Anti-Patrón	Problema	Solución
100% coverage obsession	Tests triviales que no verifican nada	Enfocarse en mutation score
Coverage como KPI de equipo	Gaming: tests vacíos	Usar como signal, no target
Ignorar coverage gaps	Code sin testear accumulate bugs	CI gate en coverage mínimo
Solo unit tests	Alto coverage, bajo valor real	Testing pyramid balanceada

Dashboard de Ejemplo



Estrategias de Mejora

- Coverage gates en CI** — Bloquear merge si coverage baja
- Delta coverage** — Solo cubrir código nuevo/changed
- Mutation testing mensual** — Ejecutar Stryker/Pitest periódicamente
- Uncovered code review** — Revisar código sin coverage regularmente
- Property-based testing** — Para algoritmos y edge cases

Conexiones

- Tasa de Defectos — Coverage previene defectos
- Deuda Técnica — Coverage bajo = tech debt
- TDD — TDD mejora coverage naturalmente
- BDD — BDD para integration tests
- Herramientas de Testing
- AI Testing Tools
- Shift Left

Technical Debt Metrics

Visión General

La deuda técnica es el costo implícito de elegir soluciones fáciles a corto plazo sobre las correctas. Medir la deuda técnica permite priorizar refactoring y prevenir que el codebase se degrade hasta volverse ingobernable.

Métricas Principales

Tech Debt Ratio (TDR)

$$\text{TDR} = (\text{Costo de remediar deuda técnica} / \text{Costo total de desarrollo}) \times 100$$

Ejemplo: 150 hours para limpiar / 2000 hours de desarrollo = 7.5% TDR

Benchmarks:

TDR	Calidad	Acción
< 5%	Excelente	Mantener
5-10%	Buena	Monitorear
10-20%	Aceptable	Planificar limpieza
> 20%	Problemática	Invertir urgentemente

Maintainability Index (MI)

$$\text{MI} = 171 - 5.2 \times \ln(\text{Volume}) - 0.23 \times (\text{Halstead Volume}) - 16.2 \times \ln(\text{LOC}) + 50 \times \sin(\sqrt{(2.4 \times \text{Cyclo})})$$

Simplificado (SonarQube):

$$\text{MI} = 206.835 - 10.1 \times (\text{HAL}) - 0.23 \times (\text{G}) - 16.2 \times \ln(\text{LOC})$$

Rango	Significado
80-100	Alta mantenibilidad
60-79	Moderada
20-59	Baja
0-19	Crítico

Cyclomatic Complexity

CC = Número de paths independientes en el programa
= Edges - Nodes + 2 (para grafos de control de flujo)

CC	Riesgo	Acción
1-10	Bajo	OK
11-20	Moderado	Refactorizar
21-50	Alto	Refactorizar urgente
> 50	Extremo	Rewrite probable

Code Smells (SonarQube)

Code Smell Count = Total de code smells detectados
Code Smell Density = Code Smells / KLOC

Categorías principales:

Categoría	Ejemplo	Impacto
Duplicación	Code copiado > 10 líneas	Mantenibilidad
Complejidad	Métodos con CC > 15	Comprensión
Naming	Variables/methods poco claros	Legibilidad
Long methods	Métodos > 50 líneas	Mantenibilidad
Large classes	Clases > 500 líneas	Acoplamiento
Dead code	Código no utilizado	Complejidad innecesaria

Debt Remediation Time

Debt Remediation = Σ (Esquemas de remediación por issue) / Horas disponibles del equipo

Ejemplo: 450 hours de deuda / 160 hours/mes = 2.8 meses para eliminar toda la deuda

Herramienta Principal: SonarQube

Métricas de SonarQube

Métrica	Descripción	Gate típico
Reliability Rating	Bugs por 1K LOC	A (0 bugs)

Métrica	Descripción	Gate típico
Security Rating	Vulnerabilidades	A (0 vulns)
Security Hotspot	Código sospechoso	0 hotspots
Maintainability Rating	Code smells	A (density < 5%)
Coverage	Cobertura de tests	> 80%
Duplications	Code duplicado	< 3%

Quality Gate de Ejemplo

- Quality Gate Conditions:
- ✓ Coverage on new code >= 80%
 - ✓ Duplicated lines on new code <= 3%
 - ✓ Maintainability Rating = A
 - ✓ Reliability Rating = A
 - ✓ Security Rating = A
 - ✓ Security Hotspots Reviewed = 100%

Dashboard de Ejemplo

TECH DEBT DASHBOARD – Julio 2026				
Tech Debt Ratio:	8.3%		△	
Maintainability:	72		OK	
Cyclomatic Avg:	8.2		OK	
Code Smells:	127	density: 4.2/KLOC		
Duplications:	2.8%		OK	
Por módulo:				
core-api:	MI=78	CC=6	✓	
frontend:	MI=65	CC=12	△	refactor needed
legacy-batch:	MI=42	CC=22	△△	critical
Remediation Time: 340h (2.1 months @ current)				
Trend: ↗ Debt growing 0.3% per sprint				

Estrategias de Gestión

Boy Scout Rule

"Dejar el campground más limpio de como lo encontraste" — refactorizar small amounts en cada PR.

Tech Debt Budget

Asignar un % fijo del sprint capacity a deuda técnica:

- **Sano:** 15-20% del capacity
- **Degradando:** 25-30%
- **Crítico:** 40%+ (considerar rewrite)

Debt Quadrants (Martin Fowler)

Deliberado	Inadvertido	
Prudent	"Hay que shippear, refactorizamos después"	"Ahora entendemos mejor el diseño"
Reckless	"No tenemos tiempo para diseño"	"¿Qué es un patrón de diseño?"

Conexiones

- Deuda Técnica - Concepto
- Refactoring Continuo
- Clean Code
- Code Coverage
- Tasa de Defectos
- Arquitectura Limpia
- IDEs (static analysis)

Team Health Metrics

Visión General

Las métricas de salud del equipo miden el bienestar, engagement y capacidad sostenible de las personas. Un equipo que no se mantiene saludable eventualmente colapsa, independientemente de sus métricas técnicas.

Métricas Principales

eNPS (Employee Net Promoter Score)

eNPS = % Promotores - % Detractores

Pregunta: "En una escala del 0-10, ¿recomendarías trabajar aquí?"

9-10 = Promotor

7-8 = Pasivo

0-6 = Detractor

Ejemplo: 45% promotores - 15% detractores = eNPS +30

Rango eNPS	Interpretación
> 50	Excelente
20-50	Bueno
0-20	Necesita atención
< 0	Crítico

Encuestas de Salud (Health Checks)

Spotify Health Check (modelo popular):

Área	Estado	Significado
Easy to release		Facilidad de deployment
Suitable for purpose		Arquitectura adecuada
Fun		Diversión y motivación
Learning		Crecimiento profesional
Mission		Claridad de objetivos
Pawns or players		Autonomía del equipo
Speed		Velocidad percibida

Área	Estado	Significado
Support		Soporte de la organización
Teamwork		Colaboración interna
Trust		Confianza entre miembros

Retention Rate

$\text{Retention Rate} = ((\text{Empleados al inicio} - \text{Bajas}) / \text{Empleados al inicio}) \times 100$

Ejemplo: $(20 - 2) / 20 = 90\%$ retention (anual)

Retention	Salud
> 90%	Excelente
80-90%	Normal
70-80%	Preocupante
< 70%	Crítico

Velocity Trends (como indicador de salud)

$\text{Velocity Trend} = \text{Velocity promedio últimos 3 sprints vs 3 anteriores}$

↗ Mejorando → Estable ↘ Preocupante (posible burnout)

Ver Velocity throughput para detalles de velocity.

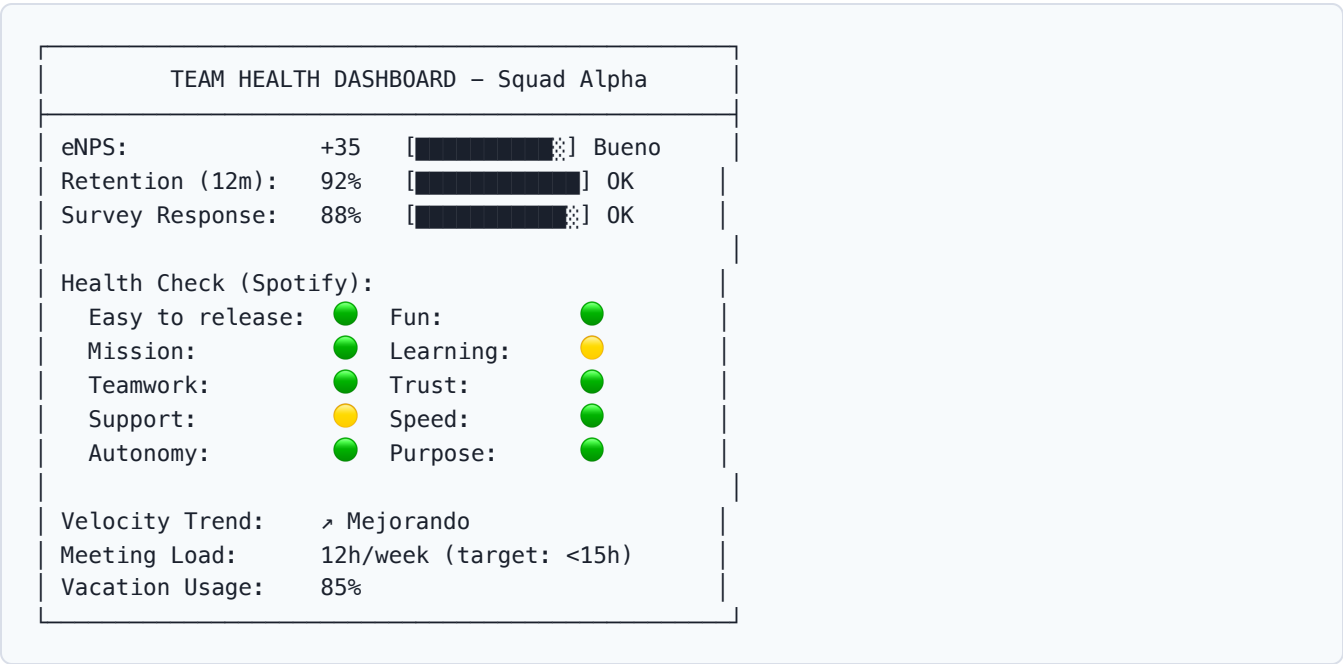
Work-Life Balance Indicators

Métrica	Cómo medir	Meta
Horas extras	Tracking de commits después de horario	< 5% del tiempo
After-hours alerts	PagerDuty alerts nocturnas	Reducing trend
Vacation usage	Días de vacaciones tomados / disponibles	> 80%
Meeting load	Horas de reunión / semana por persona	< 15h/semana

Herramientas de Survey

Herramienta	Tipo	Costo
Officevibe	Engagement platform	SaaS
15Five	Performance + surveys	SaaS
TINYpulse	Pulse surveys	SaaS
Google Forms	DIY surveys	Gratis
Slack polls	Quick checks	Gratis
Culture Amp	Enterprise engagement	Enterprise

Dashboard de Ejemplo



Estrategias de Mejora

1. **Health checks trimestrales** — Ritual regular de auto-evaluación
2. **1-on-1s semanales** — Engineering manager con cada miembro
3. **Action items con due dates** — De survey a acción concreta
4. **Transparencia de resultados** — Compartir findings con el equipo
5. **Celebrate wins** — Reconocer logros y mejora en métricas

Anti-Patrones

Anti-Patrón	Problema
Surveys sin follow-up	Desensibilización del equipo

Anti-Patrón	Problema
Usar eNPS para comparar equipos	Competencia tóxica
Ignorar feedback negativo	Loss of trust
Medir sin actuar	Peor que no medir

Conexiones

- Cultura de Equipo
- Engineering Manager
- Equipos Autónomos
- Velocity y Throughput
- Satisfacción del Cliente
- DORA Metrics
- Valor de Negocio

Customer Satisfaction

Visión General

La satisfacción del cliente mide si el software cumple las expectativas de quienes lo usan. Es el puente entre las métricas de ingeniería y el valor real de negocio. Un equipo puede tener métricas DORA perfectas y aun así construir el producto equivocado.

Las 3 Métricas Clave

NPS (Net Promoter Score)

NPS = % Promotores - % Detractores

Pregunta: "Del 0 al 10, ¿qué tan probable es que recomiendes nuestro producto?"

9-10 = Promotor

7-8 = Pasivo

0-6 = Detractor

Ejemplo: 55% promotores - 20% detractores = NPS +35

NPS Range	Interpretación
> 70	Excelente
50-70	Muy bueno
0-50	Necesita mejora
< 0	Crítico

NPS relacional vs NPS transaccional:

- **Relacional:** Midió periódicamente (trimestral), relationship con el producto
- **Transaccional:** Después de una interacción específica (soporte, features)

CSAT (Customer Satisfaction Score)

CSAT = (Satisfechos / Total respuestas) × 100

Pregunta: "¿Qué tan satisfecho estás con [feature/experiencia]?"

Escala: 1-5 (o 1-7, o emoji-based)

Ejemplo: 420 satisfechos (4-5) / 500 respuestas = 84% CSAT

CSAT	Interpretación
> 90%	Excelente
80-90%	Bueno
70-80%	Aceptable
< 70%	Preocupante

Cuándo usar CSAT vs NPS:

- CSAT → Para features específicas, transacciones puntuales
- NPS → Para percepción general del producto, relación a largo plazo

CES (Customer Effort Score)

CES = Promedio de esfuerzo percibido

Pregunta: "Qué tan fácil fue [completar tarea X]?"

Escala: 1 (muy difícil) a 7 (muy fácil)

Ejemplo: CES promedio = 5.2 (de 7)

CES	Interpretación
> 5.5	Experiencia effortless
4-5.5	Aceptable
< 4	Friction significativa

CES es el predictor más fuerte de futuras transacciones según Gartner.

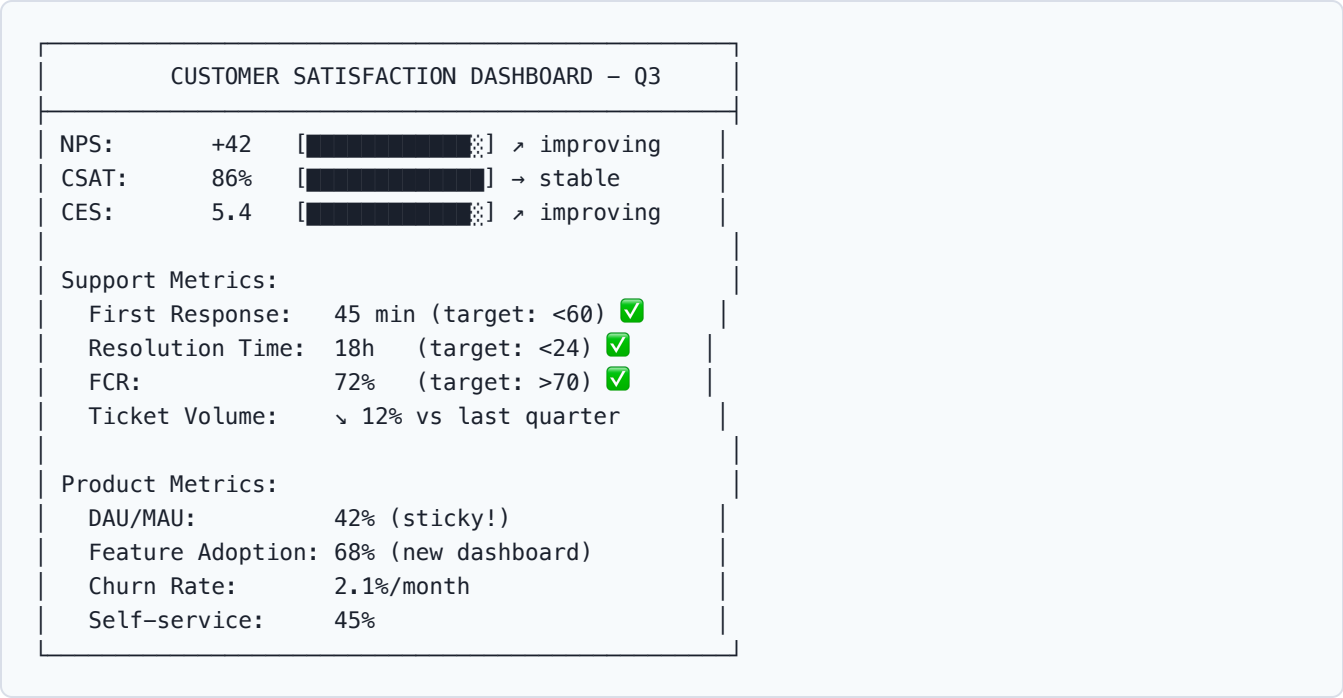
Métricas de Soporte

Métrica	Fórmula	Meta típica
First Response Time	Tiempo hasta primera respuesta	< 1 hora (email), < 5 min (chat)
Resolution Time	Tiempo hasta resolución completa	< 24 horas
First Contact Resolution	% resueltos en primer contacto	> 70%
Ticket Volume	Tickets / usuario / período	Trend ↘
Self-service Rate	% resueltos sin intervención	> 40%

Métricas de Uso del Producto

Métrica	Qué mide	Fórmula
DAU/MAU	Engagement diario/mensual	Daily active / Monthly active
Session Duration	Tiempo promedio por sesión	Total time / Sessions
Feature Adoption	% usuarios usando nueva feature	Users of feature / Total users
Churn Rate	% usuarios que se van	Lost / Starting users × 100
Retention Rate	% usuarios que vuelven	(1 – Churn) × 100

Dashboard de Ejemplo



Cómo Medir en Software

Integración con el Producto

1. **In-app surveys** — Micro-encuestas contextuales
2. **NPS emails** — Trimestrales, segmentados
3. **Support ticket analysis** — NLP para sentiment
4. **Usage analytics** — Mixpanel, Amplitude, PostHog
5. **Feedback widgets** — Canny, Nolt, ProductBoard

Segmentación

Segmento	Qué observar
Power users	Feature adoption, CES
New users	Onboarding success, first-week retention
Enterprise	NPS relacional, SLA compliance
At-risk	Churn predictors, declining usage

Conexiones

- Valor de Negocio — CSAT impacta revenue
- Salud del Equipo — Happy team → happy customers
- DORA Metrics — Mejores métricas = mejor entrega
- Product Owner — Dueño de customer satisfaction
- Definition of Done — Incluir validación de usuario
- Incident Management — Impacto en CSAT

Business Value Metrics

Visión General

Las métricas de valor de negocio conectan el trabajo de ingeniería con resultados financieros y estratégicos. Son las métricas que importan para la dirección, pero requieren colaboración entre ingeniería y negocio para medirse correctamente.

ROI de Ingeniería

Fórmula General

$$ROI = (\text{Valor generado} - \text{Costo de inversión}) / \text{Costo de inversión} \times 100$$

Ejemplo: Feature generó \$500K revenue, costó \$120K desarrollar
$$ROI = (\$500K - \$120K) / \$120K \times 100 = 316\%$$

ROI por Tipo de Proyecto

Tipo	Fórmula simplificada	Ejemplo
Revenue generation	$(\text{Incremental revenue} - \text{Dev cost}) / \text{Dev cost}$	Nueva feature premium
Cost reduction	$(\text{Annual savings} - \text{Dev cost}) / \text{Dev cost}$	Automatización de proceso
Risk mitigation	$(\text{Expected loss avoided} - \text{Dev cost}) / \text{Dev cost}$	Compliance, security
Developer productivity	$(\text{Hours saved} \times \text{hourly cost} - \text{Dev cost}) / \text{Dev cost}$	Internal tooling

Cost of Delay

$$\text{Cost of Delay} = \text{Revenue Impact per month} \times \text{Months of delay}$$

Ejemplo: Feature worth \$100K/month, 3 months delay
$$\text{Cost of Delay} = \$100K \times 3 = \$300K$$

Weighted Shortest Job First (WSJF):

$$WSJF = \text{Cost of Delay} / \text{Job Size (in story points or time)}$$

Time to Market

Definiciones

Métrica	Qué mide
Time to Market	Desde idea hasta lanzamiento
Time to First Value	Desde lanzamiento hasta primer valor
Time to Scale	Desde MVP hasta full rollout
Concept-to-Launch	Desde concepto hasta producción

Fórmula

Time to Market = Timestamp(Lanzamiento) – Timestamp(Idea aprobada)

Ejemplo: Idea aprobada 1 Marzo → Launch 15 Junio = 106 días

Benchmarking por Tipo

Tipo de proyecto	TTM típico
Feature incremental	1-4 semanas
Nueva feature completa	1-3 meses
Nuevo producto (MVP)	3-6 meses
Plataforma completa	6-18 meses

OKR Alignment (Alineación con OKRs)

Framework

```
Company OKR
├── Department OKR
│   ├── Team OKR
│   │   └── Individual KR
```

Métricas de Alineación

Métrica	Qué mide
OKR Achievement Rate	% de KRs completados (> 70% = good)
Alignment Score	% de trabajo del equipo alineado a OKRs
Output/Outcome ratio	Entregas vs resultados impactados

Alignment Score = (Horas en OKR-aligned work / Total hours) × 100

Ejemplo: 1200h en OKR work / 1600h total = 75% alignment

Revenue Impact

Métricas de Impacto

Métrica	Fórmula
Revenue per engineer	Total revenue / N° engineers
Revenue per feature	Revenue attributable / Features shipped
LTV:CAC ratio	Customer lifetime value / Acquisition cost
Expansion revenue	Revenue from existing customers / Total

Connecting Engineering to Revenue

Engineering Investment = Salaries + Tools + Infrastructure
Revenue Attribution = % of revenue directly enabled by engineering

Ejemplo: \$2M engineering investment → \$20M revenue enabled
→ Revenue per engineering dollar = \$10

Innovation Rate

Innovation Rate = (Revenue from new products/features < 12 months) / Total revenue

Ejemplo: \$3M from products < 1 year old / \$20M total = 15%

Benchmarks:

- > 20% = Alta innovación (empresas como Amazon)
- 10-20% = Saludable
- < 10% = Risk de disruption

Dashboard de Ejemplo

BUSINESS VALUE DASHBOARD – Q3 2026	
ROI (últimos 3 features):	285% avg
Time to Market (avg):	47 días (↓ vs 58)
Cost of Delay (pending):	\$420K/mes

OKR Alignment:

Company OKR 1: 82%

Company OKR 2: 70%

Company OKR 3: 55%

Revenue Metrics:

Revenue/Engineer: \$1.2M (↑ 15% YoY)

Innovation Rate: 18% (↑ vs 14% last year)

LTV:CAC: 4.2:1

Feature Revenue: \$2.1M attributable

Estrategias para Conectar Ingeniería con Negocio

1. **Regular business reviews** — Ingeniería presente en business reviews
2. **Revenue attribution** — Rastrear qué features generan revenue
3. **Cost of delay visibility** — Hacer visible el costo de no hacer
4. **OKR cascading** — Team OKRs derivan de company OKRs
5. **Post-launch reviews** — Medir impacto real vs predicho

Conexiones

- Satisfacción del Cliente — NPS/CSAT impulsan revenue
- DORA Metrics — Better delivery = faster time to market
- Balanced Scorecard — Framework integral
- Product Owner — Dueño de valor de negocio
- SAFe — Portfolio management
- Gobernanza Técnica
- Productividad con IA — AI impact en ROI

Ai Productivity Metrics

Visión General

Con la adopción masiva de herramientas de IA en software development, surgen nuevas métricas para cuantificar su impacto real. Estas métricas ayudan a determinar si la inversión en IA realmente mejora la productividad y calidad.

Métricas Principales

Acceptance Rate (Tasa de Aceptación)

$$\text{Acceptance Rate} = (\text{Sugerencias de IA aceptadas} / \text{Total sugerencias}) \times 100$$

Ejemplo: GitHub Copilot sugiere 1000 snippets, 340 aceptados = 34%

Acceptance Rate	Interpretación
> 40%	Alta utilidad, buen contexto
25-40%	Utilidad moderada
15-25%	Baja utilidad, necesita tuning
< 15%	Potencialmente contraproducente

Segmentar por tipo:

- Code completion: 30-40%
- Chat suggestions: 20-35%
- Test generation: 25-40%
- Code review: 15-30%

Time Saved (Tiempo Ahorrado)

$$\text{Time Saved} = (\text{Tasks without AI time} - \text{Tasks with AI time}) / \text{Tasks without AI time} \times 100$$

Estimación方法:

1. A/B testing: Grupo con IA vs sin IA
2. Self-reported: Encuestas a developers
3. Task timing: Medir tiempo antes/después de IA

Benchmarks de GitHub Copilot study:

Tarea	Time saved
Code writing	55%

Tarea	Time saved
Code reading	7%
Testing	10%
Overall	25-30%

Quality Impact

$$\text{Quality Impact} = (\text{Defect rate before AI} - \text{Defect rate after AI}) / \text{Defect rate before AI} \times 100$$

- 0 medir cambios en:
- Bug escape rate
 - Code review comments
 - Test coverage (delta)

Aspecto	Cómo medir	Herramienta
Code quality	SonarQube before/after	Technical debt metrics
Test quality	Mutation score delta	Code coverage
Bug rate	Bugs/sprint before/after	Defect rate
Review comments	Comments/PR trend	GitHub analytics

AI Utilization Rate

$$\text{AI Utilization} = (\text{Developers actively using AI} / \text{Total developers}) \times 100$$

Active usage = ≥ 5 AI interactions per day

ROI de AI Tools

$$\text{ROI_AI} = (\text{Time saved} \times \text{Hourly cost} - \text{AI tool cost}) / \text{AI tool cost} \times 100$$

Ejemplo: 10 devs × 5h/week saved × \$80/h × 52 weeks = \$208K saved
AI tool cost: 10 × \$20/month × 12 = \$2,400
ROI = (\$208K - \$2.4K) / \$2.4K × 100 = 8,567%

Dashboard de Ejemplo

AI PRODUCTIVITY DASHBOARD – Julio 2026
--

Acceptance Rate:	34%		OK
Time Saved (avg):	28%		OK
AI Utilization:	85%		Great
Quality Impact:	↗ Improving (↓ bugs 15%)		

Por herramienta:

Copilot:	Acceptance 36%, Time saved 31%
AI Review:	Acceptance 28%, Comments ↓ 22%
AI Testing:	Acceptance 42%, Coverage ↑ 8%
AI Chat:	Acceptance 31%, Adoption 78%

Monthly Trend:

Acceptance:	Jan 22% → Jun 34% ↗
Time saved:	Jan 15% → Jun 28% ↗
Quality:	Stable to slightly improving

Financial:

Monthly AI cost:	\$2,400
Estimated savings:	\$40K/month
ROI:	1,567%

Methodologies de Medición

1. A/B Testing

Grupo A (control): Desarrollo sin AI tools
Grupo B (treatment): Desarrollo con AI tools

Medir: Time to complete, defects, satisfaction
Duración: 4-8 semanas

2. Pre/Post Adoption

Período pre (3 meses): Métricas antes de AI
Período post (3 meses): Métricas después de AI

Medir: Delta en velocity, cycle time, defects
Controlar: Team composition changes

3. Developer Surveys

Monthly pulse:

- "How useful was AI this month?" (1-5)
- "What tasks did AI help with?"
- "Any concerns about AI-generated code?"

Categorías de AI Impact

Categoría	Herramientas	Métricas clave
Code completion	Copilot, Cody, Codeium	Acceptance rate, time saved
Code review	CodeRabbit, Qodo	Comments reduced, bugs caught
Testing	Diffblue, Mabl	Coverage increase, test speed
Documentation	AI doc generators	Doc coverage, time saved
Architecture	AI assistants	Design decisions, tech debt

Anti-Patrones

Anti-Patrón	Problema
Assuming AI = better	No medir = no saber si funciona
Ignoring code quality	AI-generated code puede ser peor
Over-reliance	Skills atrophy si se usa IA para todo
Not segmenting	Aceitar todo no mide valor real

Conexiones

- IA en Software Engineering
- AI Coding Assistants
- AI Code Review
- AI Testing Tools
- Velocity y Throughput
- Deuda Técnica
- Valor de Negocio

Deployment Frequency

Visión General

Deployment Frequency es una de las 4 DORA metrics y mide con qué frecuencia el equipo despliega código a producción. Es un proxy de la velocidad de delivery y la madurez del pipeline CI/CD.

Definición y Medición

Fórmula

Deployment Frequency = N° deploys exitosos a producción / Período

Ejemplos:

- 120 deploys / mes = 6/día → ELITE
- 20 deploys / mes = 1/2 días → HIGH
- 4 deploys / mes = 1/sem → MEDIUM
- 1 deploy / mes → LOW

Qué Contar

Incluir	No incluir
Deploys a producción	Deploys a staging/dev
Feature releases	Hotfixes (separar)
Configuration changes	Database migrations (separar)
Infrastructure updates	Rollbacks (contar como separate event)

Fuentes de Datos

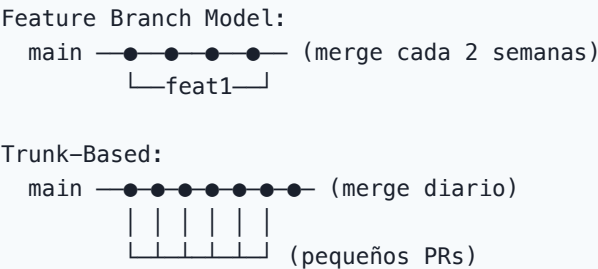
Fuente	Cómo extraer
GitHub Actions	workflow_run events for deploy workflows
Jenkins	Deploy job completion timestamps
ArgoCD	Sync events
Kubernetes	kubectl rollout history
AWS CodeDeploy	Deployment history API
Vercel/Netlify	Deployment API

Niveles de Desempeño

Nivel	Frecuencia	Características
Elite	On-demand, múltiples/día	CI/CD completo, feature flags, trunk-based
High	Entre semanal y mensual	CI/CD maduro, test automation
Medium	Entre mensual y semanal	CI básico, deployment manual parcial
Low	Menos que mensual	Deploy manual, fear-based

Estrategias de Mejora

1. Trunk-Based Development



Ver Trunk-Based Development.

2. Feature Flags

```
# Deploy sin feature visible
if feature_flag("new_checkout", user):
    show_new_checkout()
else:
    show_old_checkout()
```

Desacoplar **deploy** de **release** permite desplegar más frecuentemente sin riesgo.

Ver Feature Flags.

3. CI/CD Pipeline Completo

Stage	Herramienta	Automatización
Build	GitHub Actions, Jenkins	100%
Test	Jest, pytest, JUnit	100%
Security scan	Snyk, Trivy	100%

Stage	Herramienta	Automatización
Deploy staging	ArgoCD, Spinnaker	100%
Deploy prod	ArgoCD, Spinnaker	100% (con approval)
Monitoring	Prometheus, Grafana	100%

4. Reduce Deployment Batch Size

Menor batch size → Menor riesgo → Mayor frecuencia

Deploy 1 feature → Risk bajo → Deploy frecuente
Deploy 20 features → Risk alto → Deploy infrecuente

Deployment Frequency por Tipo

Tipo	Frecuencia típica	Ejemplo
Hotfix	On-demand	Bug crítico en producción
Feature deploy	Diario-semanal	Nueva funcionalidad
Infrastructure	Semanal-mensual	Kubernetes upgrade
Configuration	On-demand	Feature flag change

Dashboard de Ejemplo

DEPLOYMENT FREQUENCY DASHBOARD	
Current:	4.2 deploys/día [ELITE]
Trend:	↗ Mejorando (3.1 → 4.2 en 3 meses)
Breakdown:	
Features:	2.8/día [██████████░░░░░░░░]
Hotfixes:	0.3/día [████░░░░░░░░░░░░░░]
Config:	0.8/día [██████████░░░░░░░░]
Infra:	0.3/día [████░░░░░░░░░░░░░░]
Pipeline metrics:	
Build time:	4.2 min (P50)
Test time:	8.5 min (P50)
Deploy time:	1.2 min (P50)
Total pipeline:	14 min
Success rate: 98.5% (1.5% require rollback)	

Avg WIP deploys: 3.2 (parallel deploys)

Métricas Relacionadas

Métrica	Conexión	
[[12-change-failure-rate\	Change Failure Rate]]	Calidad de cada deploy
[[13-mean-time-recovery\	MTTR]]	Qué pasa cuando falla
[[03-cycle-time\	Cycle Time]]	Velocidad del flujo
[[05-code-coverage\	Code Coverage]]	Quality gate antes de deploy
[[09-business-value-metrics\	Time to Market]]	Impacto en entrega de valor

Referencias

- DORA Metrics
- CI/CD Pipeline
- Trunk-Based Development
- Feature Flags
- Release Management
- Herramientas CI/CD

Change Failure Rate

Visión General

Change Failure Rate (CFR) es una de las 4 DORA metrics. Mide el porcentaje de cambios a producción que resultan en un fallo de servicio (degradation, rollback o hotfix). Es el indicador clave de la calidad del proceso de deployment.

Definición y Medición

Fórmula

$$CFR = (\text{Cambios que causan fallo} / \text{Total cambios}) \times 100$$

Ejemplo: 5 fallos en 100 deploys = 5% CFR → ELITE

Qué cuenta como "fallo"

Evento	Incluir	Ejemplo
Rollback inmediato	Sí	Deploy revertido < 1 hora
Hotfix	Sí	Fix crítico dentro de 24h
Degradación	Sí	Alerta de monitoreo activada
Data corruption	Sí	Pérdida o corrupción de datos
Partial outage	Sí	Feature unavailable
Performance degradation	Sí	Latencia > SLA

Qué NO contar

Evento	Razón
Failed CI build (before deploy)	No llegó a producción
Feature flag disable	No es deployment failure
Scheduled maintenance	Esperado, no es fallo
Third-party outage	Fuera de control

Niveles de Desempeño

Nivel	CFR	Interpretación
Elite	0-15%	Muy bajo riesgo por cambio
High	16-30%	Riesgo moderado
Medium	16-30%	Similar a high (varía por contexto)
Low	16-30%	Riesgo inaceptable

Segmentación

Por Tipo de Cambio

CFR por tipo:

Feature deploy:	8%		OK
Bug fix:	3%		Great
Hotfix:	15%		Acceptable
Config change:	2%		Great
Infrastructure:	12%		OK

Por Complejidad

Complejidad	CFR típico	Acción
Simple (config)	< 5%	OK
Moderate (feature)	5-15%	Monitorear
Complex (arch change)	15-25%	Extra QA
High risk (migration)	25-40%	War room

Estrategias de Reducción

1. Testing Automation

Before: Manual testing → 25% CFR
After: Automated testing suite → 8% CFR

Investment: 3 months, 2 engineers
Payback: 17% reduction in failures × cost per failure

Ver Code Coverage para métricas de testing.

2. Progressive Rollout

Canary deploy:

1% traffic → 5% → 25% → 50% → 100%

Si falla en 1%, impacto es mínimo

Reduce CFR significativamente

3. Automated Rollbacks

```
# ArgoCD rollback config
strategy:
  canary:
    steps:
      - setWeight: 5
      - pause: {duration: 5m}
      - analysis:
          templates:
            - templateName: success-rate
          args:
            - name: service-name
              value: my-service
    rollback:
      enabled: true
      failureThreshold: 3
```

4. Quality Gates

Gate	Qué verifica	Herramienta
Unit tests pass	Lógica core	Jest, JUnit
Integration tests pass	APIs, DB	Testcontainers
Security scan clean	Vulnerabilities	Snyk, Trivy
Performance test pass	Latencia, throughput	k6, Gatling
Code review approved	Calidad de código	GitHub PRs

5. Feature Flags

Deploy con flag OFF → Test in production safely → Enable gradually

Reduce riesgo de cada deployment a casi cero.

Análisis de Causa Raíz

Cuando CFR aumenta, investigar:

- Tasa de Defectos — Defectos en producción
- CI/CD Pipeline — Automatizar para reducir CFR
- Change Management
- Feature Flags

Mean Time Recovery

Visión General

Mean Time to Recovery (MTTR) mide el tiempo promedio que toma un sistema en recuperarse de un fallo en producción. Es una de las 4 DORA metrics y refleja la capacidad del equipo para detectar, diagnosticar y resolver incidentes.

Definición y Medición

Fórmula

$$MTTR = \Sigma(\text{Tiempo de recuperación por incidente}) / N^{\circ} \text{ incidentes}$$

Ejemplo:
Incident 1: 45 min recovery
Incident 2: 120 min recovery
Incident 3: 30 min recovery
 $MTTR = (45 + 120 + 30) / 3 = 65 \text{ min}$

Componentes del MTTR

$$MTTR = \text{Detection Time} + \text{Diagnostic Time} + \text{Remediation Time} + \text{Recovery Verification}$$

|← Detection →|← Diagnostic →|← Remediation →|← Verification →|
| Alert fires | Team gathers | Fix deployed | Service restored|

Componente	Qué es	Benchmark
Detection time	Tiempo hasta que alguien se entera	< 5 min (con monitoreo)
Diagnostic time	Tiempo para encontrar la causa	< 15 min
Remediation time	Tiempo para aplicar fix	< 30 min
Verification time	Tiempo para confirmar que funciona	< 10 min

Métricas Relacionadas

Métrica	Diferencia con MTTR
MTTD (Detection)	Solo el tiempo de detección
MTTDiag (Diagnostic)	Solo el tiempo de diagnóstico
MTTA (Acknowledgment)	Tiempo hasta que alguien responde

Métrica	Diferencia con MTTR
MTBF (Between Failures)	Tiempo promedio entre fallos

Niveles de Desempeño

Nivel	MTTR	Características
Elite	< 1 hora	Auto-remediation, runbooks, on-call efectivo
High	< 1 día	Good monitoring, response procedures
Medium	< 1 día	Basic monitoring, ad-hoc response
Low	1 día - 1 semana	Poor observability, manual processes

Estrategias de Mejora

1. Mejorar Detección (↓ MTTD)

Estrategia	Impacto	Herramienta
Alerting inteligente	Alto	PagerDuty, Opsgenie
SLO-based alerts	Alto	Custom dashboards
Anomaly detection	Medio	ML-powered monitoring
Synthetic monitoring	Medio	Pingdom, Checkly

Ver Observabilidad.

2. Mejorar Diagnóstico (↓ MTTDiag)

Estrategia	Impacto
Distributed tracing	Alto
Structured logging	Alto
Runbooks automatizados	Alto
War room procedures	Medio
Status pages	Medio

3. Mejorar Remediación (↓ Remediation Time)

Estrategia	Impacto
Automated rollbacks	Muy alto

Estrategia	Impacto
Feature flag disable	Alto
Blue/green deploys	Alto
Chaos engineering	Medio
Game days	Medio

Ver Chaos Engineering.

4. Incident Response Process

1. DETECT

→ Alert fires → Pager notification

2. ACKNOWLEDGE

→ On-call responds → < 5 min

3. TRIAGE

→ Assess severity → Assign severity level

4. DIAGNOSE

→ Find root cause → Runbook if available

5. REMEDIATE

→ Fix/rollback → Verify recovery

6. REVIEW

→ Post-incident review → Action items

Incident Severity Matrix

Severidad	Descripción	Response SLA	Recovery SLA
SEV1	Service down, data loss	5 min	1 hora
SEV2	Major feature broken	15 min	4 horas
SEV3	Minor feature degraded	1 hora	24 horas
SEV4	Cosmetic, workaround exists	4 horas	1 semana

Dashboard de Ejemplo

MTTR DASHBOARD - Julio 2026

MTTR (Overall):

38 min

ELITE

MTTR (SEV1):

22 min

ELITE

MTTR (SEV2):

45 min

ELITE

MTTR (SEV3):

2.1h

HIGH

Breakdown (avg):

Detection:

3 min

Acknowledgment:

5 min

Diagnosis:

12 min

Remediation:

15 min

Verification:

3 min

Incidents (30 days):	
Total:	8
SEV1:	1 (MTTR: 22 min)
SEV2:	3 (MTTR: 45 min)
SEV3:	4 (MTTR: 2.1h)
MTBF (Mean Time Between Failures): 3.8 days	
Auto-remediated: 3/8 (37.5%)	

Auto-Remediation Patterns

Pattern	Ejemplo	Impacto en MTTR
Auto-rollback	Kubernetes rollback on health check fail	↓ 90%
Circuit breaker	Hystrix, Envoy	↓ 80%
Auto-scaling	HPA, VPA	↓ 70%
Self-healing	Restart on OOM	↓ 60%

Post-Incident Review Template

```
## Incident: [Title]
- **Date**: YYYY-MM-DD
- **Duration**: X hours Y minutes
- **Severity**: SEV1/2/3/4
- **MTTR**: X minutes

### Timeline
- HH:MM - Alert fired
- HH:MM - On-call acknowledged
- HH:MM - Root cause identified
- HH:MM - Fix deployed
- HH:MM - Service verified recovered

### Root Cause
[Description]

### Action Items
- [ ] [Owner] [Action] [Due date]
- [ ] [Owner] [Action] [Due date]

### Lessons Learned
[What went well, what didn't]
```

Conexiones

- DORA Metrics — MTTR es una de las 4 key metrics
- Change Failure Rate — Fallos que causan recovery
- Deployment Frequency — Recovery = rollback
- Incident Management
- Observabilidad
- Monitoring Tools
- Satisfacción del Cliente

Balance Scorecard

Visión General

El Balanced Scorecard (BSC), adaptado para software engineering, proporciona un marco integral para medir el desempeño desde 4 perspectivas balanceadas. Evita la optimización local de una métrica a costa de otras.

Las 4 Perspectivas

FINANCIERA
ROI · Cost per feature · Revenue per engineer
CLIENTE
NPS · CSAT · CES · Feature adoption
PROCESOS INTERNOS
DORA · Cycle time · CFR · Quality gates
APRENDIZAJE Y CRECIMIENTO
eNPS · Training hours · Innovation rate

Perspectiva 1: Financiera

KPIs

KPI	Fórmula	Meta típica
ROI de Ingeniería	$(\text{Revenue attributable} - \text{Cost}) / \text{Cost}$	> 200%
Cost per Feature	$\text{Dev cost} / \text{Features shipped}$	Decreasing trend
Revenue per Engineer	$\text{Total revenue} / \# \text{ engineers}$	Growing YoY
Infrastructure Cost Ratio	$\text{Infra cost} / \text{Revenue}$	< 15%
Innovation Investment	% budget en new vs maintenance	20-30% new

Dashboard

FINANCIERA:

ROI: 285% ↗

Cost/Feature: \$12K (↓ from \$15K)

Rev/Engineer: \$1.2M (↑ 15%)

Infra Ratio: 11% 
Innovation: 22% 

Ver Business value metrics para detalles.

Perspectiva 2: Cliente

KPIs

KPI	Fórmula	Meta típica
NPS	%Promotores – %Detractores	> 40
CSAT	Satisfechos / Total × 100	> 85%
CES	Promedio esfuerzo (1-7)	> 5.0
Feature Adoption	Users of feature / Total users	> 60%
Time to Value	Time from launch to first value	< 30 días

Dashboard

CLIENTE:
NPS: +42 ↗
CSAT: 86%
CES: 5.4
Adoption: 68%
Time to Value: 18 days

Ver Customer satisfaction para detalles.

Perspectiva 3: Procesos Internos

KPIs

KPI	Fórmula	Meta típica
Deploy Frequency	Deploys / time	> 1/día
Lead Time	P50 commit→prod	< 1 día
Change Failure Rate	Failures / deploys × 100	< 15%
MTTR	Avg recovery time	< 1 hora
Cycle Time	P50 dev time	< 3 días
Test Coverage	Covered lines / total × 100	> 80%

KPI	Fórmula	Meta típica
Tech Debt Ratio	Remediation / dev cost × 100	< 10%

Dashboard

PROCESOS INTERNOS:

Deploy Freq:	4.2/día	[ELITE]
Lead Time:	4.2h	[ELITE]
CFR:	8.5%	[ELITE]
MTTR:	38 min	[ELITE]
Cycle Time:	2.3d	[GOOD]
Coverage:	87%	[GOOD]
Tech Debt:	8.3%	[ACCEPTABLE]

Ver Dora metrics, Cycle time, Deployment frequency, Change failure rate, Mean time recovery.

Perspectiva 4: Aprendizaje y Crecimiento

KPIs

KPI	Fórmula	Meta típica
eNPS	%Promotores – %Detractores	> 30
Retention Rate	(Start – Leaves) / Start × 100	> 90%
Training Hours	Hours per engineer per quarter	> 20h
Innovation Rate	New product revenue / total	> 15%
AI Adoption	Active AI users / total engineers	> 80%
Knowledge Sharing	Internal talks / month	> 2

Dashboard

APRENDIZAJE Y CRECIMIENTO:

eNPS:	+35 ↗
Retention:	92%
Training:	24h/qtr
Innovation:	18%
AI Adoption:	85%
Knowledge:	3 talks/mes

Ver Team health metrics, Ai productivity metrics.

Implementación del BSC

Paso 1: Seleccionar KPIs (3-5 por perspectiva)

No intentar medir todo. Elegir las métricas más impactables.

Paso 2: Establecer Baseline

Medir durante 2-3 sprints para tener baseline.
Comparar después de establecer el baseline.

Paso 3: Definir Targets

Target Type	Ejemplo
Absolute	MTTR < 1 hora
Relative	CFR ↓ 20% vs baseline
Directional	eNPS ↗ quarter over quarter

Paso 4: Review Cadence

Review	Frecuencia	Participantes
Operational	Weekly	Team leads
Tactical	Monthly	Engineering managers
Strategic	Quarterly	VP/Director + stakeholders

Paso 5: Iterate

Cada quarter:

1. Review BSC achievements
2. Adjust targets
3. Add/remove KPIs based on context
4. Present to stakeholders

BSC de Ejemplo: Squad Alpha Q3 2026

BALANCED SCORECARD – SQUAD ALPHA Q3		
FINANCIERA	Target	Actual
ROI:	>200%	285%
Cost/Feature:	<\$15K	\$12K

CLIENTE			
NPS:	>40	+42	✓
Feature Adoption:	>60%	68%	✓
PROCESOS			
Deploy Freq:	>1/d	4.2	✓
CFR:	<15%	8.5%	✓
MTTR:	<60m	38m	✓
APRENDIZAJE			
eNPS:	>30	+35	✓
Retention:	>90%	92%	✓
Training:	>20h	24h	✓
OVERALL STATUS: 100% ✓			

Conexiones

- DORA Metrics — Perspectiva de procesos
- Valor de Negocio — Perspectiva financiera
- Satisfacción del Cliente — Perspectiva cliente
- Salud del Equipo — Perspectiva de crecimiento
- Modelos de Madurez
- Gobernanza Técnica
- SAFe

README

Visión General

Las métricas son el sistema nervioso de una software factory. Sin medición no hay mejora posible: lo que no se mide, no se puede gestionar. Esta sección cubre las métricas más relevantes para evaluar el desempeño de equipos, procesos y productos de software.

Principios Clave

- 1. **Medir para mejorar, no para castigar** — Las métricas deben驱动 improvement, no performance reviews
- 2. **Outcome over output** — Preferir métricas de resultado sobre métricas de actividad
- 3. **Context matters** — Ninguna métrica tiene sentido sin contexto de negocio
- 4. **Leading over lagging** — Métricas predictivas son más útiles que las retrospectivas
- 5. **Automatizar la recolección** — La métrica manual es inútil a largo plazo

Mapa de Métricas

Métricas de Flujo y Entrega

Métrica	Archivo	Foco
DORA Metrics	Dora metrics	4 key metrics del Accelerate State of DevOps
Velocity y Throughput	Velocity throughput	Story points vs throughput, limitaciones
Cycle Time y Lead Time	Cycle time	Tiempo de desarrollo y entrega
Deployment Frequency	Deployment frequency	Frecuencia de despliegues
Change Failure Rate	Change failure rate	Tasa de fallos en cambios
Mean Time to Recovery	Mean time recovery	Tiempo de recuperación

Métricas de Calidad

Métrica	Archivo	Foco
Tasa de Defectos	Defect rate	Defect density, escaped defects
Code Coverage	Code coverage	Cobertura de tests, mutation testing
Deuda Técnica	Technical debt metrics	SonarQube, code smells, maintainability

Métricas de Personas y Negocio

Métrica	Archivo	Foco
Salud del Equipo	Team health metrics	eNPS, retention, velocity trends
Satisfacción del Cliente	Customer satisfaction	NPS, CSAT, CES
Valor de Negocio	Business value metrics	ROI, time-to-market, OKRs

Métricas Especializadas

Métrica	Archivo	Foco
Productividad con IA	Ai productivity metrics	Acceptance rate, time saved
Balanced Scorecard	Balance scorecard	Framework integral 4 perspectivas

Framework de Métricas

BUSINESS VALUE ROI · Time-to-Market · OKR Alignment
CUSTOMER SATISFACTION NPS · CSAT · CES
TEAM HEALTH eNPS · Retention · Flow State
DELIVERY CAPABILITY DORA · Cycle Time · Deployment Freq
CODE QUALITY Coverage · Defect Rate · Tech Debt

Cómo Usar Esta Sección

- 1. **Implementando DORA** → Dora metrics como punto de partida
- 2. **Evaluando calidad** → Defect rate + Code coverage
- 3. **Midendo equipo** → Team health metrics con encuestas
- 4. **Alineando con negocio** → Balance scorecard
- 5. **Incorporando IA** → Ai productivity metrics
- 6. **Definiendo dashboard** → Combinar métricas de arriba

Anti-Patrones Comunes

Anti-Patrón	Problema	Solución
Vanity metrics	Números que suenan bien pero no impulsan acción	Medir outcomes, no outputs
Metric fixation	Optimizar la métrica en vez del valor	Usar métricas como signal, no target
Goodhart's Law	"Cuando una métrica se convierte en target, deja de ser buena métrica"	Combinar múltiples métricas
Measure what's easy	Medir solo lo que es fácil de capturar	Invertir en telemetría adecuada

Referencias

- DevOps Filosofía
- Modelos de Madurez
- CI/CD Pipeline
- Observabilidad
- Definition of Done
- Herramientas de Monitoreo
- IA en Software